

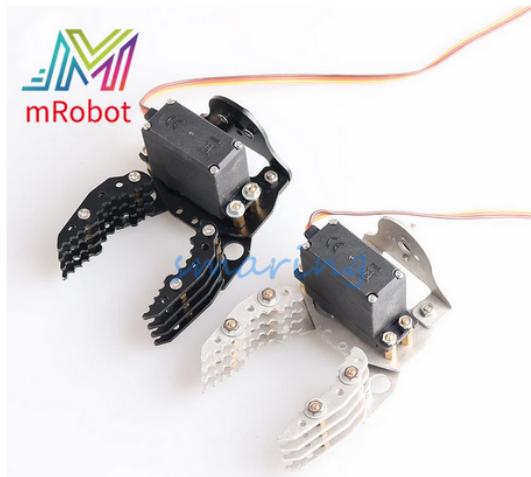
Fecha: 13/03/2023

Alumnos: Dragos Ivan Cornel, en colaboración con Santiago y Julián

Memoria para la práctica de pinza robótica

1. INTRODUCCIÓN

En esta práctica se está trabajando para hacer una pinza robótica adecuada para el brazo robótico. Para ello se ha comprado el siguiente pack de materiales de Aliexpress, con las siguientes características:



Pinza de brazo de Metal para Arduino, pinza mecánica con Servo RC de alto Torque, pieza robótica educativa, bricolaje, 150mm
6 vendidos

€ 11,48 - 23,24 ~~€ 12,09 - 24,46~~ -5%

Precio con IVA incluido

Color:

Cantidad:

— 1 + 5388 unidades disponibles

Envío a  Spain

Tipo de artículo: Model

Género: Unisex

Dimensiones: other

Control remoto: Si

Franja de edad: >8 años

Número de modelo: sm

Estado: Artículos en stock

Escala: other

Accesorios para soldados: Piezas y componentes del soldado

Paquete original: YES

Origen: CN(Origen)

Advertencia: other

Número de serie del fabricante: Robot

Tipo de versión: Primera edición

Tipo de producto básico: Asunto

Fuente de inspiración: CHINA

Grado de finalización: Producto semiacabado

Tema: Robots

Material: Metal

Tipo de marioneta: Model

Especificaciones:

Material: aleación de aluminio duro

Peso: aproximadamente 40g (sin servo)

Separación angular máxima: 86mm

Longitud total: 83mm (la longitud total más larga cuando la pata está cerrada).

Anchura total ①: 150mm (anchura máxima cuando la pata está abierta).

Anchura total ②: 55mm (anchura máxima cuando la pata está cerrada).

Grosor total: 54mm (grosor máximo total con patas servo).

El paquete incluye:

Astilla sin servo = 1 x pinza G6 (sin servo)

Plata con MG996R = 1 x G6 pinza + 1 x MG996R servo

Plata con DS3218 = 1 x G6 pinza + 1 x DS3218 servo

El paquete negro es el mismo que el paquete plateado, solo el color de la pinza es diferente.

2. CÁLCULO Y SIMULACIÓN EN PROTEUS

En el intento que vamos a ver a continuación, la simulación aparentemente había resultado exitosa. Pero no fue hasta montarlo en la protoboard que nos dimos cuenta de que la pinza no seguía los pulsos según los siguientes cálculos que habíamos usado para el montaje:

$$T_a \rightarrow 2'34 \text{ ms} \cdot 10^{-3} = (R_a + 25K) \cdot 100 \cdot 10^{-9}$$

a) High time $\overset{\text{constante}}{=} 0'69 \cdot (R_1 \cdot C)$

b) Low time $= 0'69 \cdot (R_2 \cdot C)$

a) $2'34 \cdot 10^{-3} = 0'69 \cdot (R_1 \cdot 100 \cdot 10^{-9}) \rightarrow \frac{2'34 \cdot 10^{-3}}{0'69 \cdot 100 \cdot 10^{-9}} = R_1$

$2'34 \cdot 10^{-3} = 0'69 R_1 + 6'9 \cdot 10^{-8}$

$0'69 R_1 = (2'34 \cdot 10^{-3}) - (6'9 \cdot 10^{-8})$

$0'69 R_1 = 2'34 \cdot 10^{-3}$

$R_1 = \frac{2'34 \cdot 10^{-3}}{0'69} = 3'39 \cdot 10^{-3}$

$R_1 = 33\,766\,\Omega$

Cerrar:

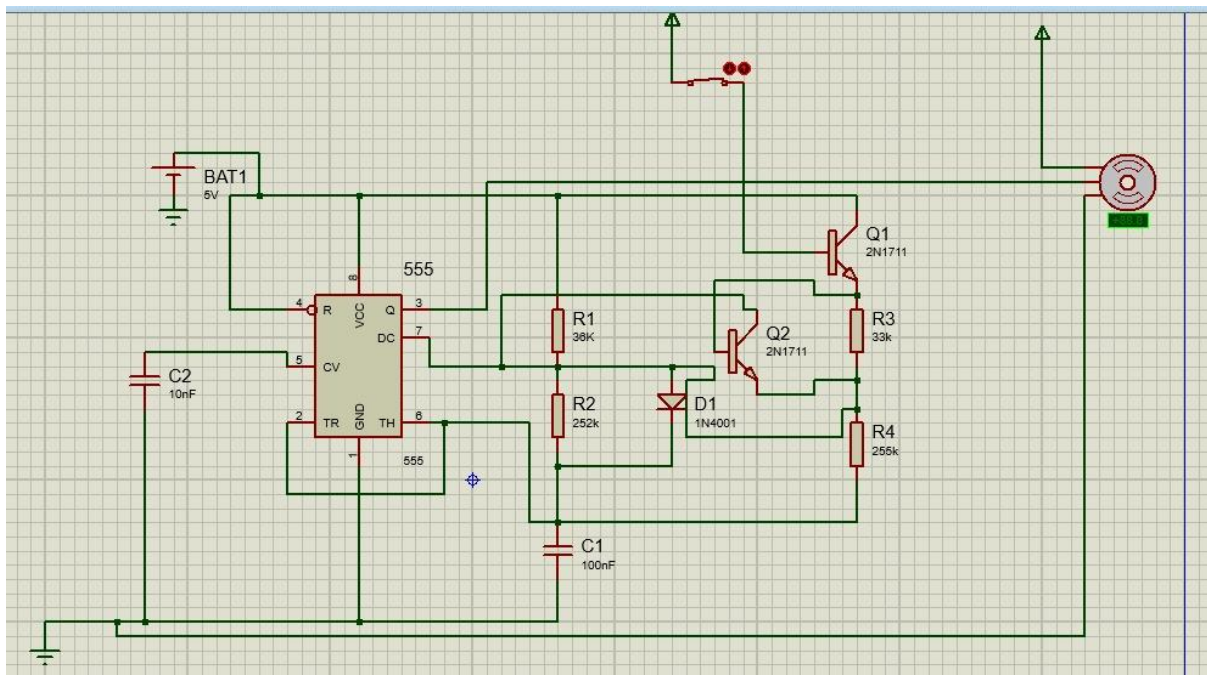
$$R_1 = 36\,075\,\Omega$$

$$R_2 = \frac{17'5 \cdot 10^{-3}}{0'693 \cdot 100 \cdot 10^{-9}} = 252\,525'25\,\Omega$$

Abrir:

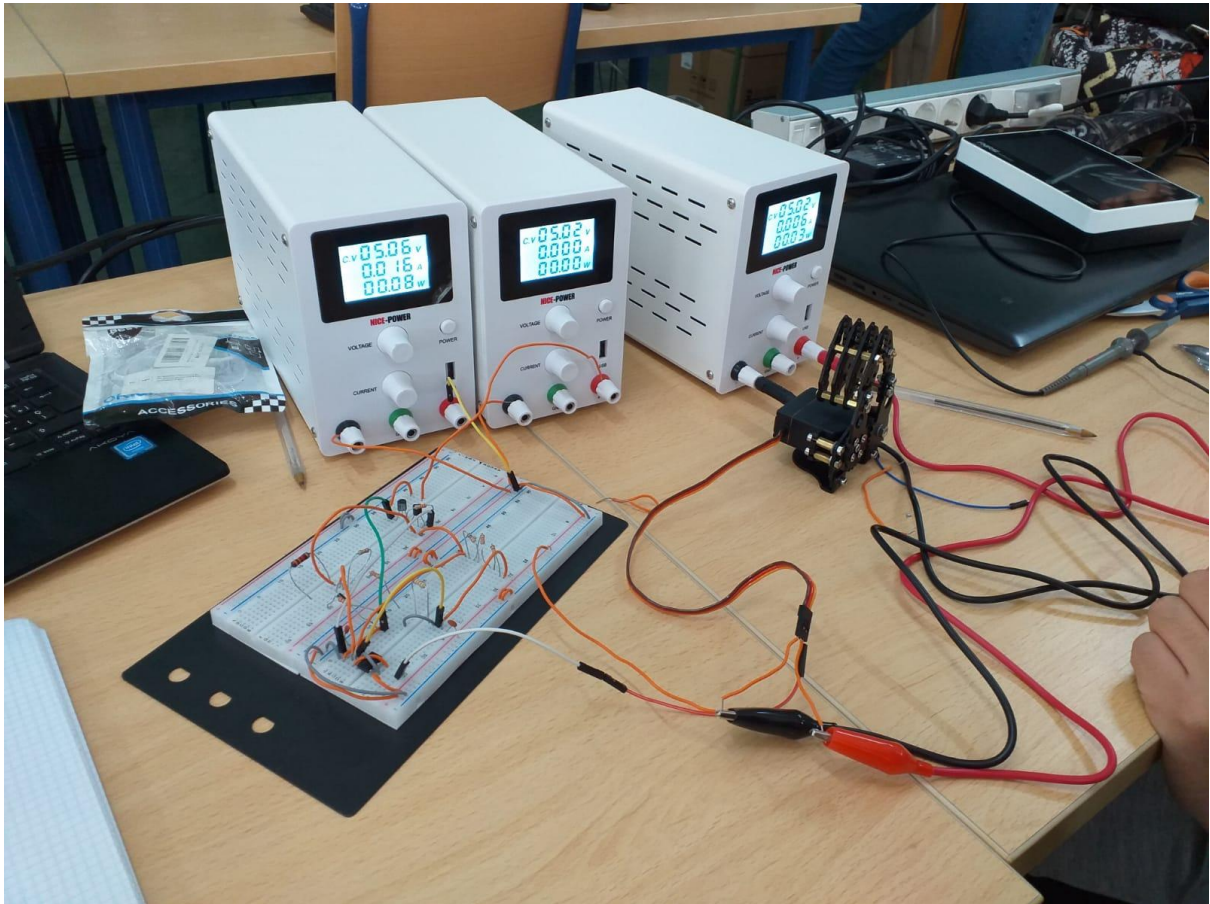
$$R_1 = 33\,766\,\Omega$$

$$R_2 \Rightarrow \frac{17'7 \cdot 10^{-3}}{0'693 \cdot 100 \cdot 10^{-9}} = 255\,411'25\,\Omega$$



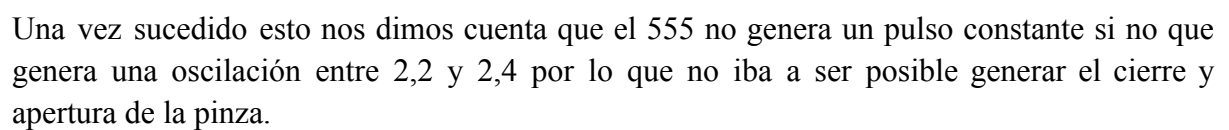
Según este montaje en Proteus, por tanto, la pinza reaccionaba de manera que con un 0 lógico se cerraba pero no con las características que nosotros estábamos esperando: un periodo de 17,5 ms y un pulso de 2,5ms para cerrarse, y 17,7ms y 2,3 ms para abrirse. Sino que midiendo con el osciloscopio, nos salió un periodo de 16ms aprox y un pulso de 4 ms, todo ello constante e invariablemente de los cambios en el switch de 1 o 0. Por tanto no había un

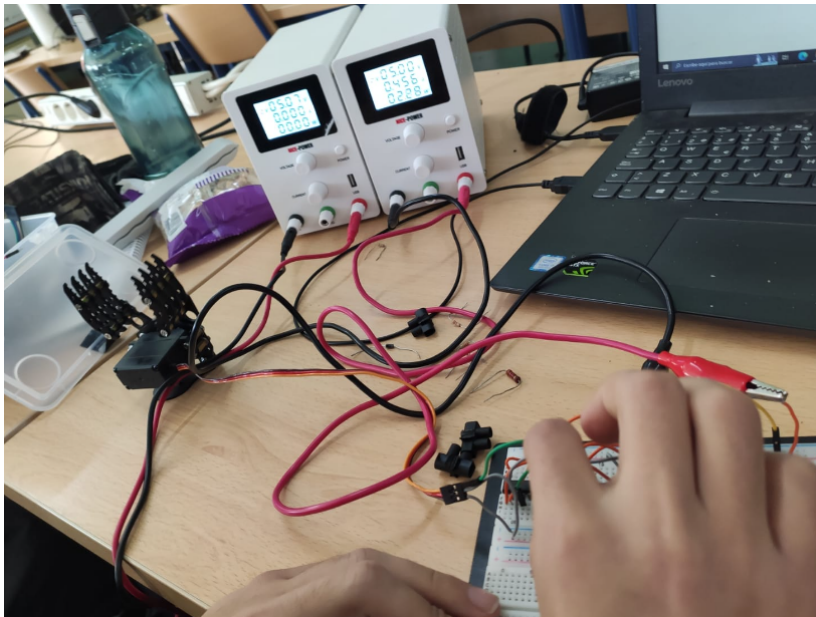
valor oscilante resultante del 555, sino un valor constante que solo permitía que se cerrase la pinza, apretando sin límite.



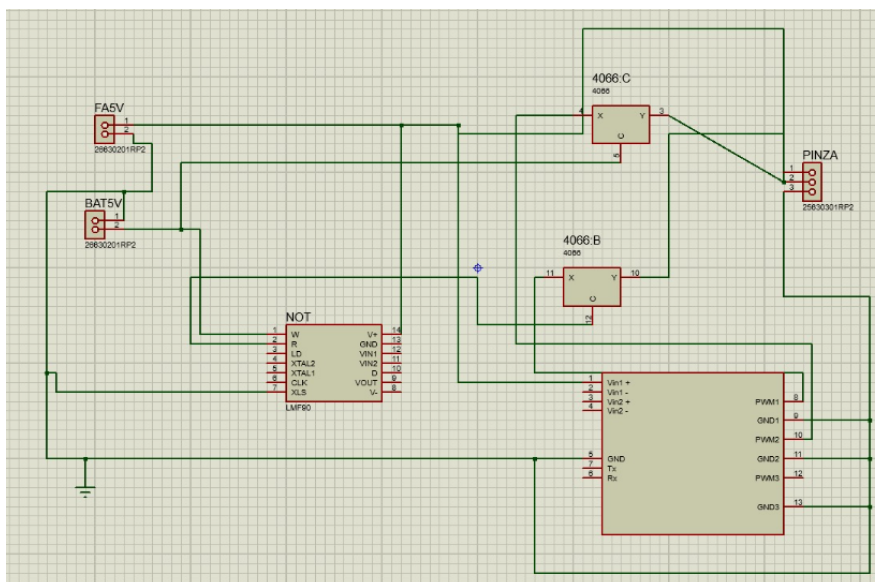
Tiempo después nos dimos cuenta de que uno de los condensadores no era de 100nF, sino de 50nF.

Hemos vuelto a montar el circuito en Proteus y en la protoboard, con el condensador adecuado, y sin embargo la pinza sigue recibiendo ruido. Volveremos a montar el circuito en físico para localizar y eliminar el ruido.

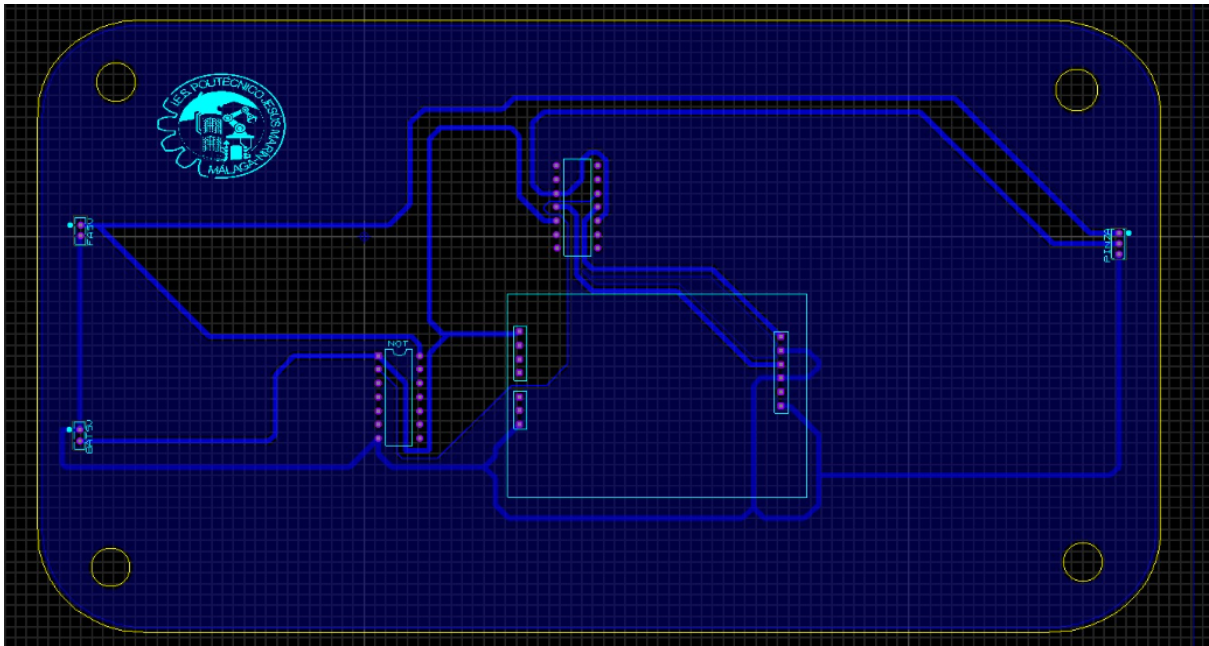




En este [video](#) podemos observar el efecto mencionado anteriormente.



Captado el problema en el anterior montaje, decidimos desechar el 555 y sumar al circuito anterior un generador de pulsos PWM el cual se conecta al 4066 y su salida a la pinza. Haciendo esto logramos tener los pulsos que calculamos con los que la pinza se abre y cierra con la precisión que necesitamos. Realizamos el montaje físico y comprobamos que este circuito si funciona correctamente por lo que comenzamos a realizar las medidas de los componentes y crear nuestra PCB. En resumen, de ahí el temblor de la pinza, porque con el 555 no se generaba un pulso constante, sino alguno entre 2.2 ms y 2.4 ms. Con este cambio, gracias a la implementación del 4066 ya si se establecía firmemente cuando sale un 1 o un 0 hacia la pinza.



Aquí se muestra el diseño de nuestra PCB, con la cual tuvimos que crear los componentes del generador de pulsos y se dieron algunos problemas para generar el plano de masa ya que los bordes de la placa no estaban bien conectados entre sí.

Solucionado estos generamos el archivo Gerber para enviarlos a China.

3. PINZA SOLO CON EL ESP32

Además del proyecto realizado con componentes físicos realizamos el mismo funcionamiento de la pinza pero utilizando directamente los pulsos de PWM del ESP32. Para ello hemos tenido que realizar una programación.

En la programación debemos añadir una señal de entrada la cual va a ser introducida por el brazo robótico y va a indicar cuando debe abrirse y cerrarse la pinza

En principio hicimos una programación la cual constaba con dos estados uno de apertura y otro de cierre. Esta programación funcionaba en la simulación pero al montarlo físicamente presentaba un error en la apertura de la pinza ya que solo abre cuando se le desconectaba la alimentación.

A continuación dejo un enlace a la simulación mencionada.

<https://wokwi.com/projects/356275772247482369>

```
#include <Servo.h>
#define servopin 9
#define senal 4
boolean abre = false;
boolean cierra = false;
```

```
Servo myservo;
int servopos = 180; //para la posición del servo, 180 o 120
grados
int servopos2 = 120;

void pinza()
{
    if (digitalRead(senal) == 0 && !abre) {
        abre = true;
        cierra = false;
    }
    if (digitalRead(senal) == 1 && !cierra) {
        cierra = true;
        abre = false;
    }
    if (abre==true ){
        myservo.write(servopos);

    }
    if(cierra==true){
        myservo.write(servopos2);

    }
}

void setup() {
    myservo.attach(servopin);
    pinMode(senal, INPUT_PULLUP);
    Serial.begin(9600);

}

void loop()
{
```

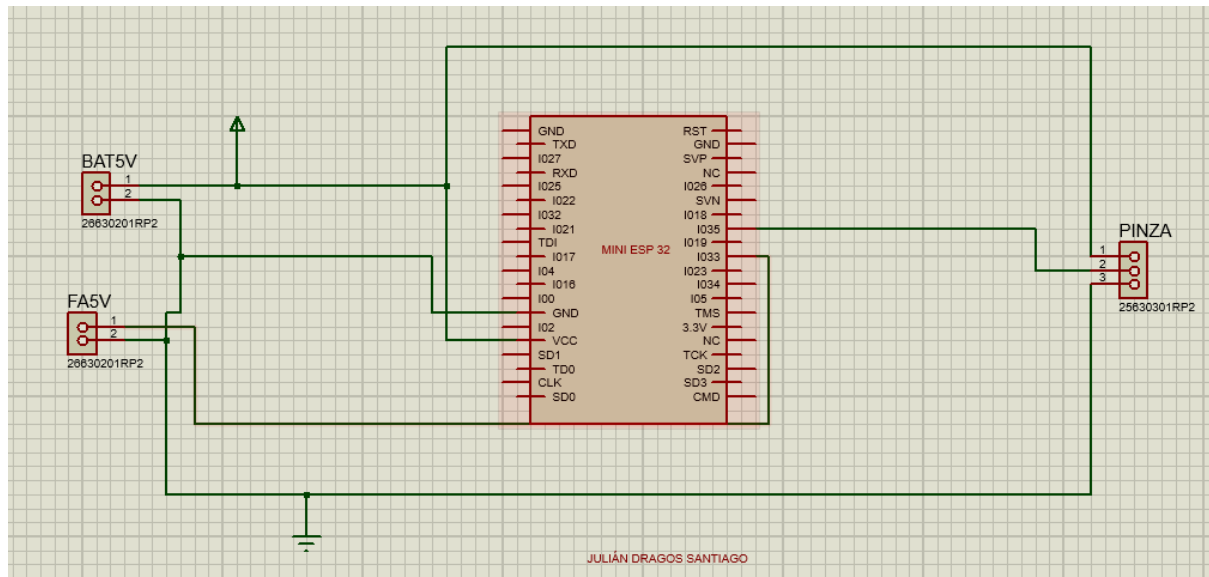
```
pinza();  
}
```

Dados los problemas con la anterior programación hemos realizado una nueva pero utilizando un solo estado. Mediante ese estado le hemos programado que si la señal es negativa abre y si es positiva cierra.

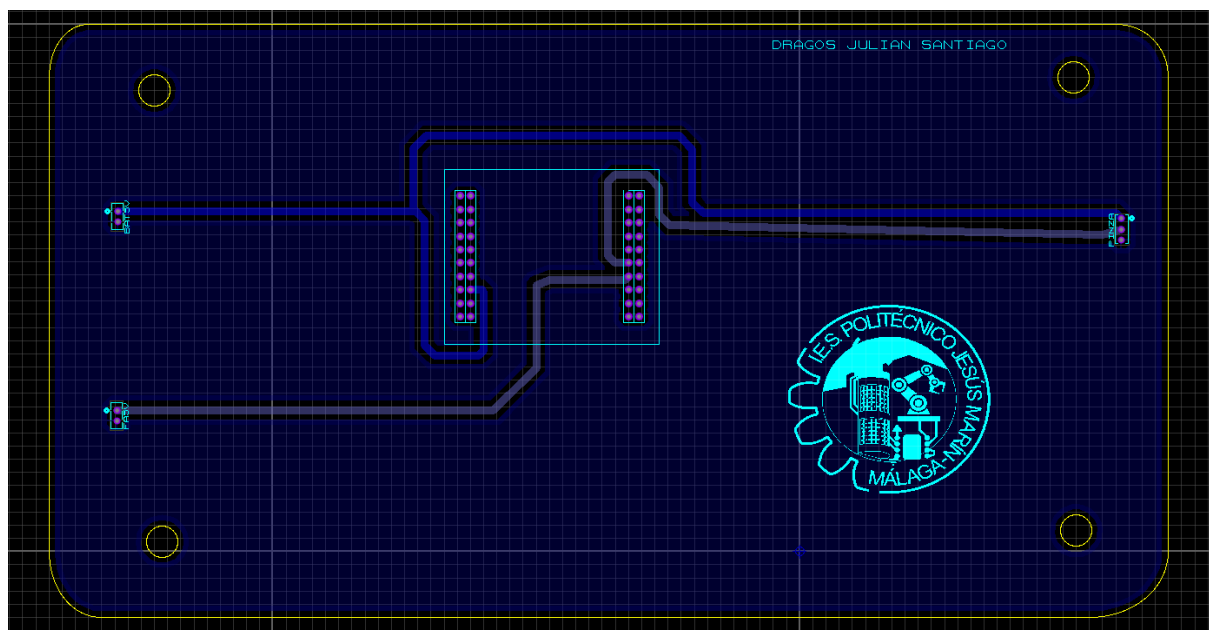
```
#include <ESP32Servo.h>  
#define servopin 13  
#define senal 12  
boolean abierta;  
  
Servo myservo;  
  
void setup() {  
  pinMode(senal, INPUT);  
  pinMode(servopin, OUTPUT);  
  myservo.attach(servopin);  
  abierta = false;  
}  
  
void loop()  
{  
  if (digitalRead(senal) == LOW) {  
    abierta = true;  
  }  
  else  
  {  
    abierta = false;  
  }  
  
  if (abierta)  
  {  
    myservo.write(180);  
  }  
  else  
  {  
    myservo.write(120);  
  }  
}
```


}

Esquema eléctrico:



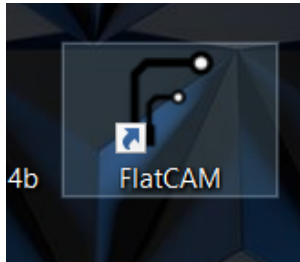
PCB:



Para realizar el esquema eléctrico y la PCB tuvimos que crear el ESP32 tomando las medidas físicas de la que utilizaremos en la placa.

4. Fabricación de PCB con CNC

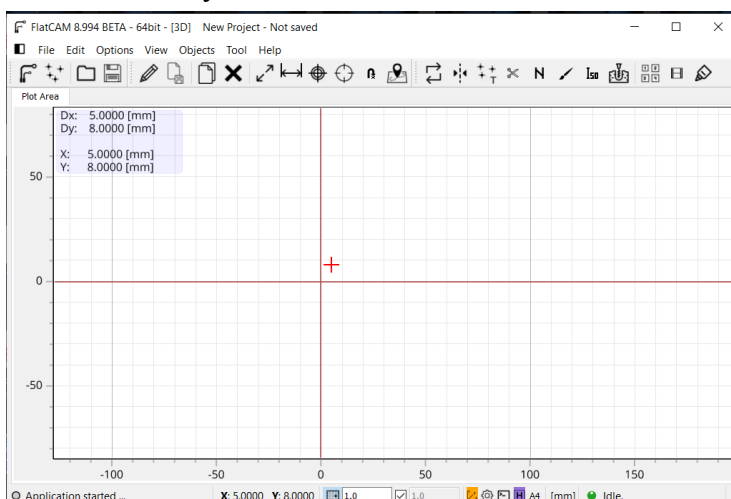
En esta parte hemos utilizado 2 programas, el primero es FlatCam(versión 8.9), este programa carga los archivos gerber y excellon (obtenidos anteriormente mediante Proteus) y genera el código G-Code y este código contiene la tarea necesaria para poder fabricar el prototipo. Estas son Aislamiento, Taladros y Corte.



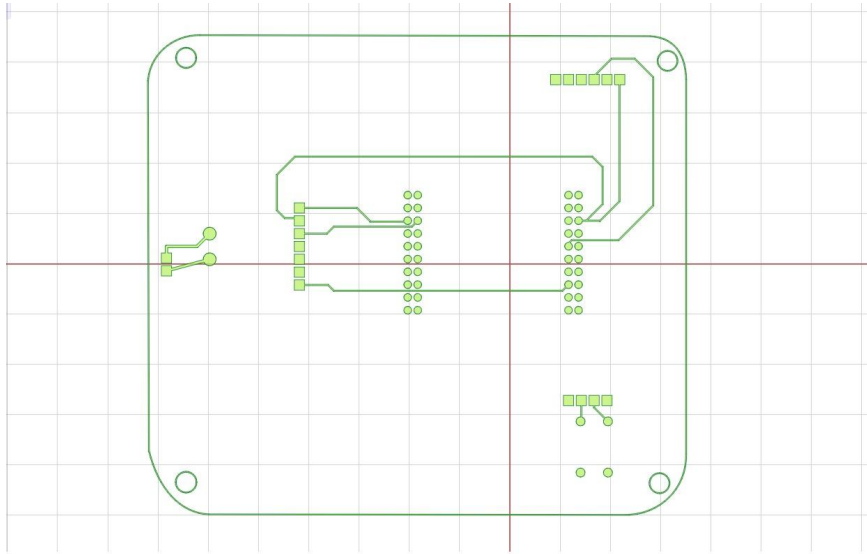
La otra aplicación que usaremos será el Candle GRBL, este hace conexión directa con el Arduino de la máquina para su manejo. Lo que hace es cargar los archivos G-Code(obtenidos en FlatCam) con los trabajos de enrutado en tres ejes (X,Y,Z).



Una vez abrimos FlatCam, nos sale una pantalla de inicio en la cuál tenemos que exportar el archivo Gerber y Excellon.



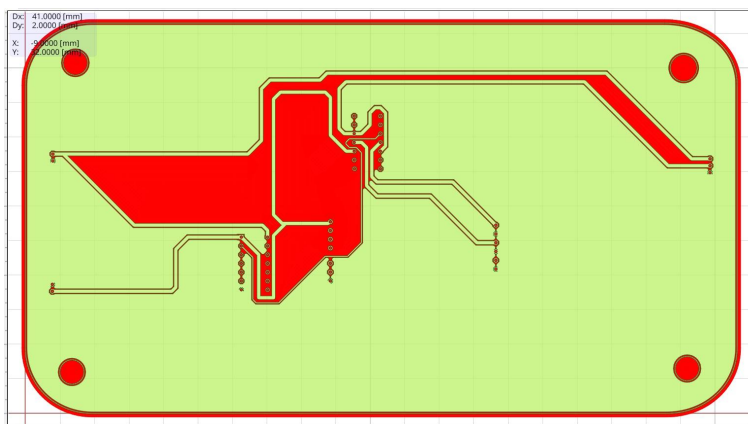
Cuando cargamos Gerber y Excellon, se exporta el archivo de corte.



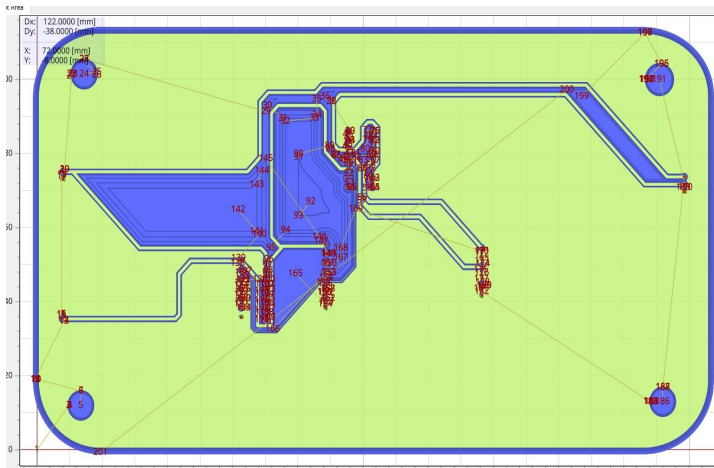
Lo siguiente sería reducir la escala 0.1 de los taladros, modificar el origen para que tanto X como Y estén en 0.

Lo siguiente que hay que hacer es hacer “Mirror” del plano, es decir, realizar un giro espejo del plano en los 3 archivos. Este volteo se puede hacer tanto sobre el eje X como sobre el eje Y, lo importante es que todos han de ser el mismo.

Una vez hecho estos pasos, hay que realizar el cálculo de herramienta, esto lo podemos hacer en el menú de funciones extras, en la calculadora, indicamos los datos técnicos de la herramienta de grabado en V, añadimos la herramienta que hemos calculado escribiendo el valor en forma de V y pulsando en “search and add”. Ponemos los parámetros (Overlap, Method, Margin), y seleccionamos “Generate Geometry”.



El programa nos genera un archivo de geometría en el cuál se puede observar los trazados que va a realizar la fresa.



Esto sería el trabajo de fresado, es decir, un archivo G-Code con el trabajo que va a realizar. (Tenemos que exportarlo).

Generamos archivos G-Code de la sección de agujeros que se va a realizar con bit de 1mm, 1,2mm y 3 mm y acto seguido exportamos drill y hole, por lo tanto ahora tenemos un archivo G-Code de los trabajos de taladros de 0.8, 1, 1.2mm.

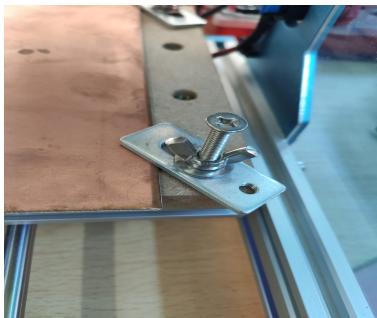
El siguiente paso es crear los archivos de corte de PCB.

Una vez tenemos todo lo anterior, seleccionamos el archivo gerber de corte y sobre este archivo, cambiando el diámetro de la fresadora, los parámetros de tarea(CutZ, Multi-Depth y Gap size), además de los espacios sin cortar, generamos el archivo G-Code y exportamos el trabajo de corte.

Después pasamos al trabajo en la CNC, vamos a utilizar el programa candle mencionado anteriormente, lo primero que debemos de saber es que para poder conectarnos a la CNC desde el ordenador, debe estar conectada mediante un cable de USB a USB tipo B.

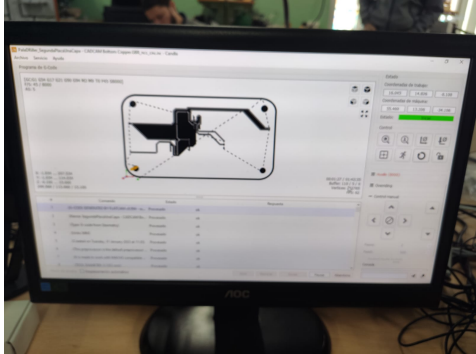
Para empezar le hemos colocado la punta de 1.2mm para hacer los bordes de la placa y los agujeros, colocamos la herramienta en V de limpieza de cobre para que haga eso (limpiar el cobre), situamos el cabezal en los orígenes XY y le damos a recordar o guardar los puntos.

Conectamos la sonda Z. Para la parte negativa nos ayudamos de los sujetadores de la placa



Y para el positivo, lo conectamos en la punta de la herramienta.

Al pulsar origen de trabajo Z, la broca empieza a bajar hasta que se choca con la placa y al estar en contacto el positivo con el negativo, la broca rechaza la placa y sube unos centímetros, acto seguido vuelve a bajar lentamente hasta chocarse con la placa de nuevo, guardamos origen en Z y ese va a ser el punto en el que va a empezar a trabajar.

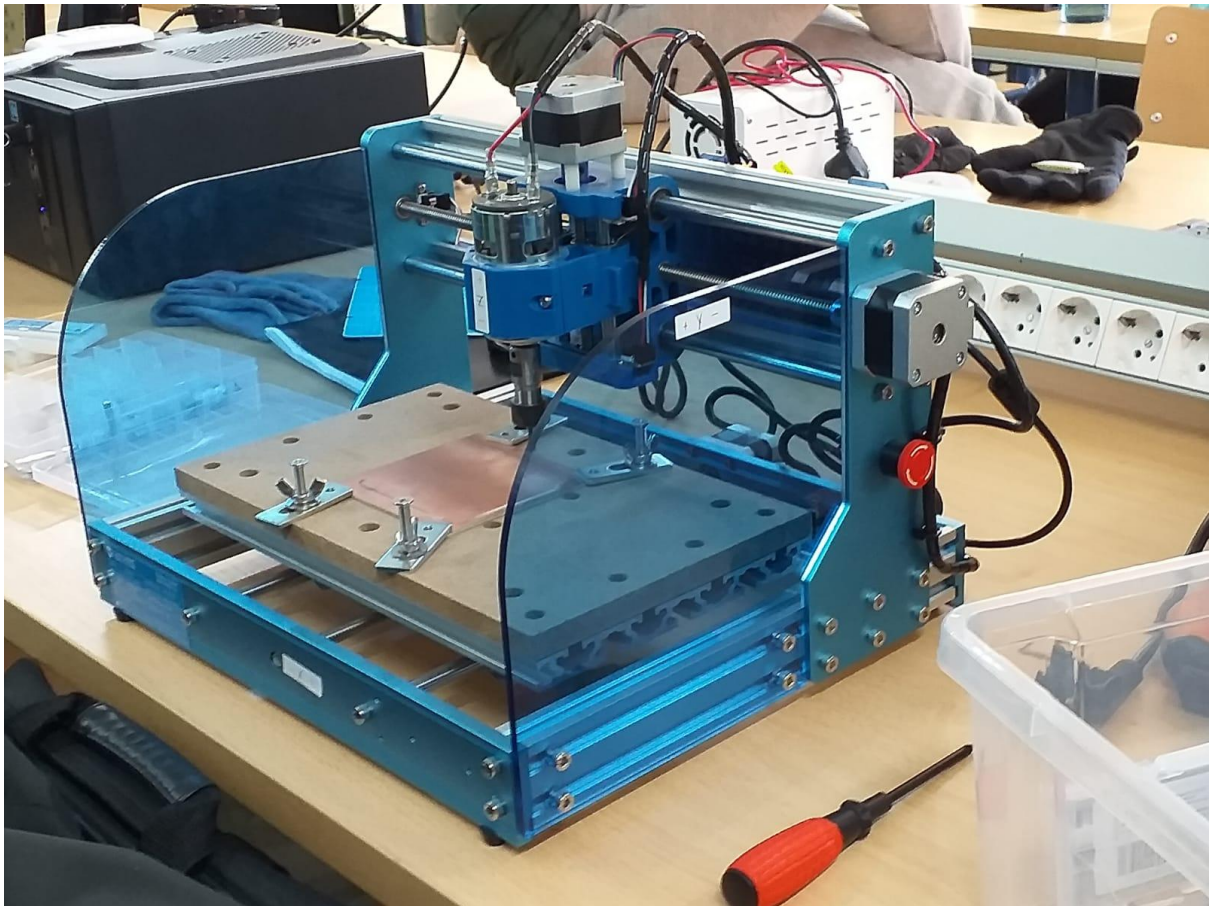


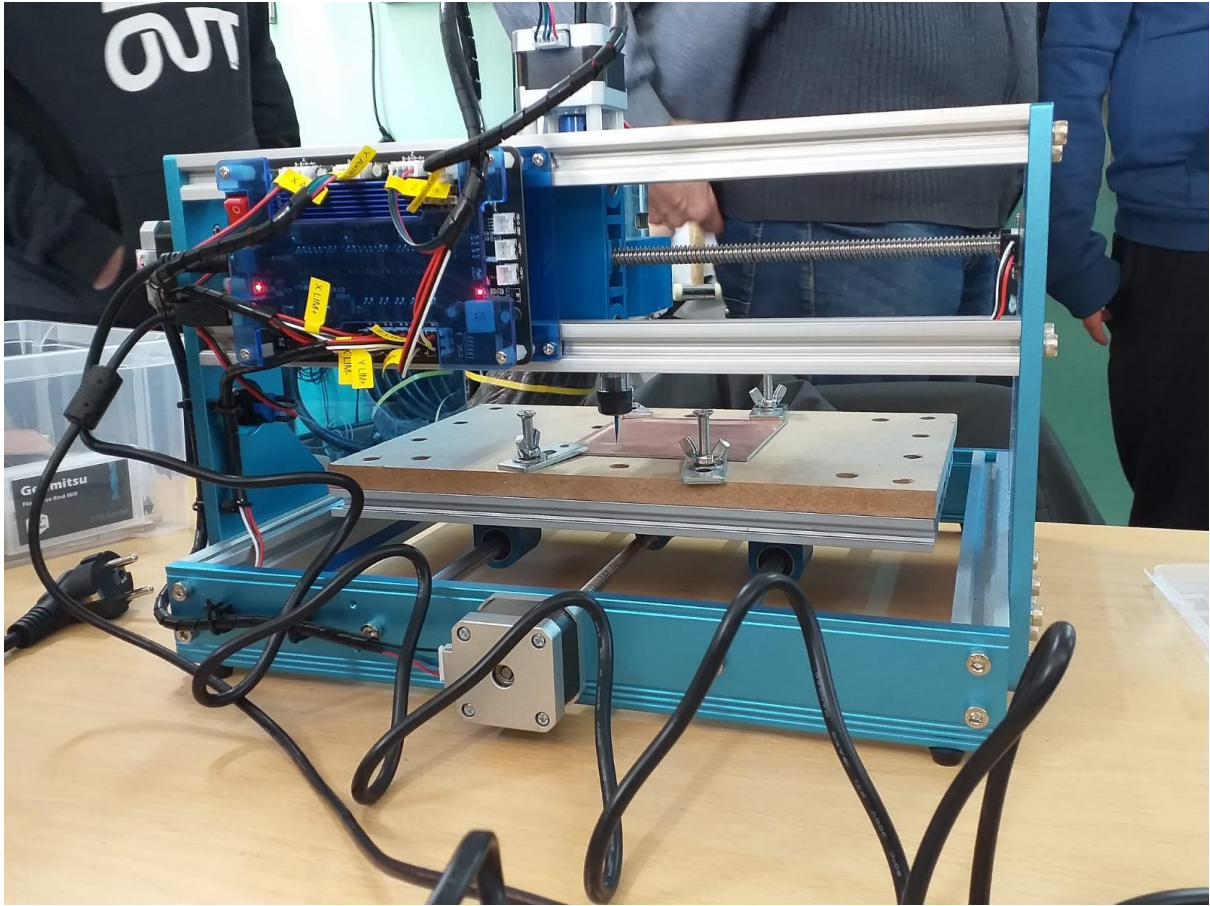
El tiempo que tarda en terminar esa parte del trabajo puede variar en función del tamaño de la placa y del modo de trabajo (NCC o insolation).

En nuestro caso, nos ha ocurrido 2 veces lo mismo, en mitad de este primer proceso de la broca de 1.2mm (bordes y agujeros), se ha parado la CNC y 2 placas se nos han quedado de esta forma:



Una vez el trabajo de la broca de 1.2 mm ha finalizado, debemos de cambiar la broca a la de 1mm o 0.8mm (depende de la parte que vayamos a realizar).





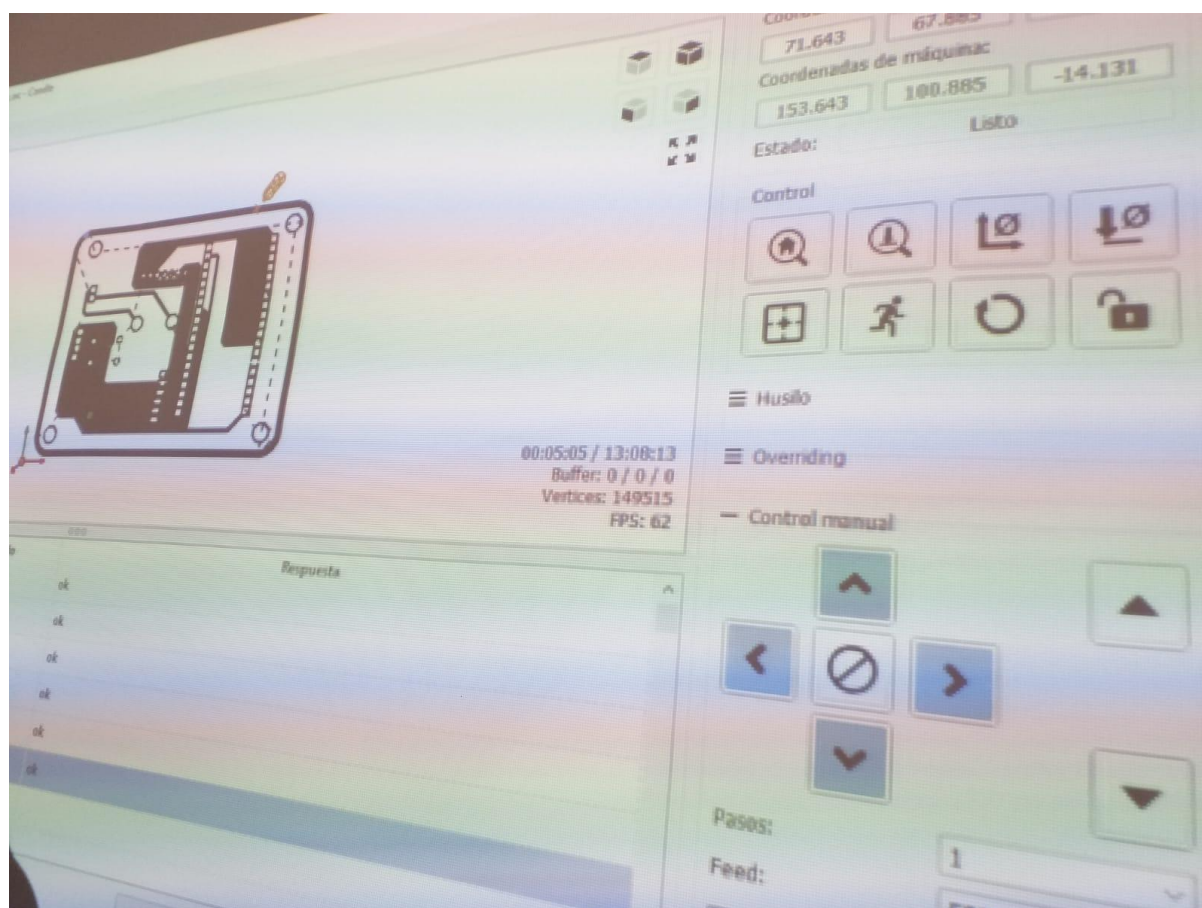
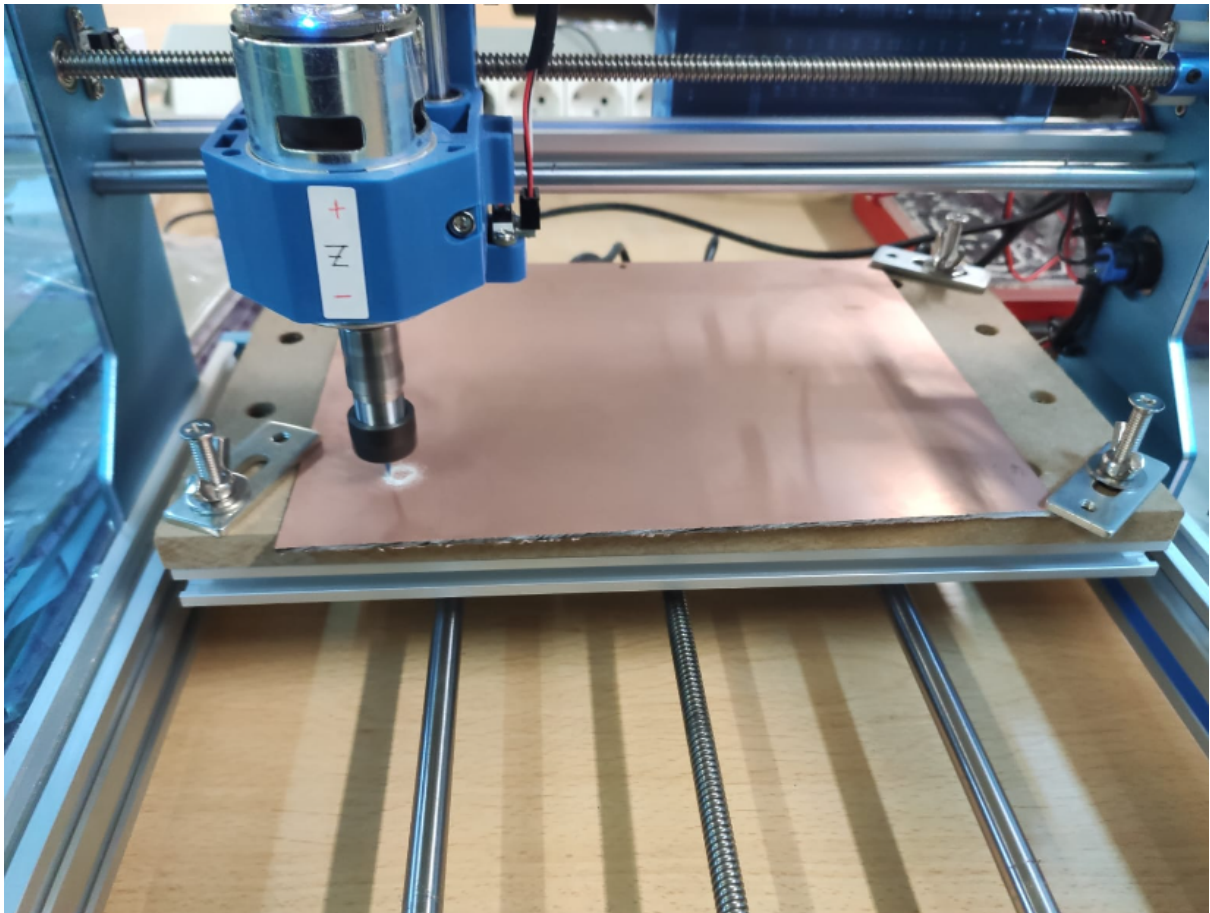
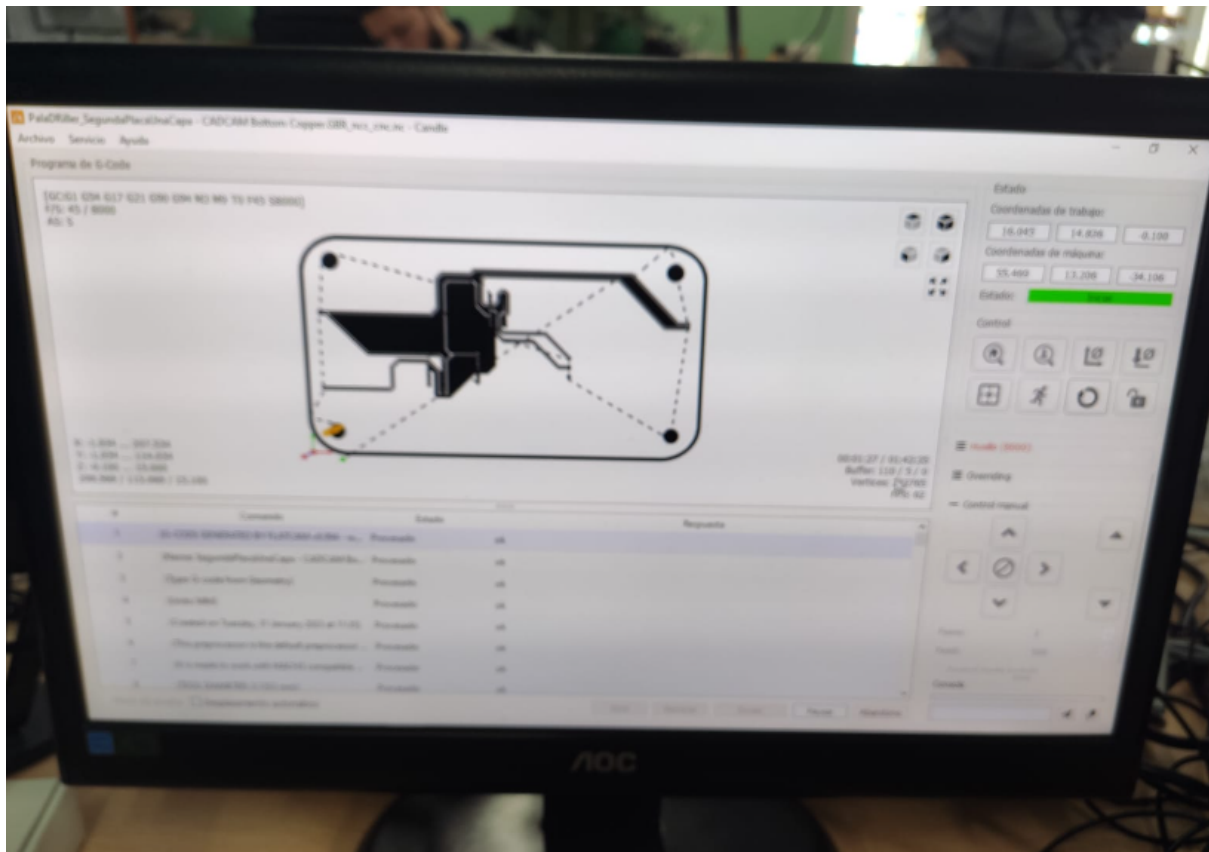


Imagen donde se ve el seguimiento de la impresión “en directo”.



Si nos damos cuenta, nos hemos encontrado con un pequeño fallo, porque no hemos comprobado que la placa PCB tenga el tamaño esperado. Por tanto, al haber impreso la placa, nos dimos cuenta que la dimensión era x4 veces más grande de lo esperado.



En esta etapa final, acabaron de llegar las placas de China con los archivos Gerber que en principio parecían correctos (para el ojo novato), pero como veremos, encontraremos un conjunto de fallos, muchos de ellos arrastrados desde el segundo 1 por falta de experiencia. Esperemos que esta sea una gran oportunidad para aprender una buena lección sobre el funcionamiento de Proteus y el cuidado del diseño. Esto lo supimos en su momento, pero no fuimos capaces de encontrar solución por ningún lado, porque tampoco es fácil describir el problema.

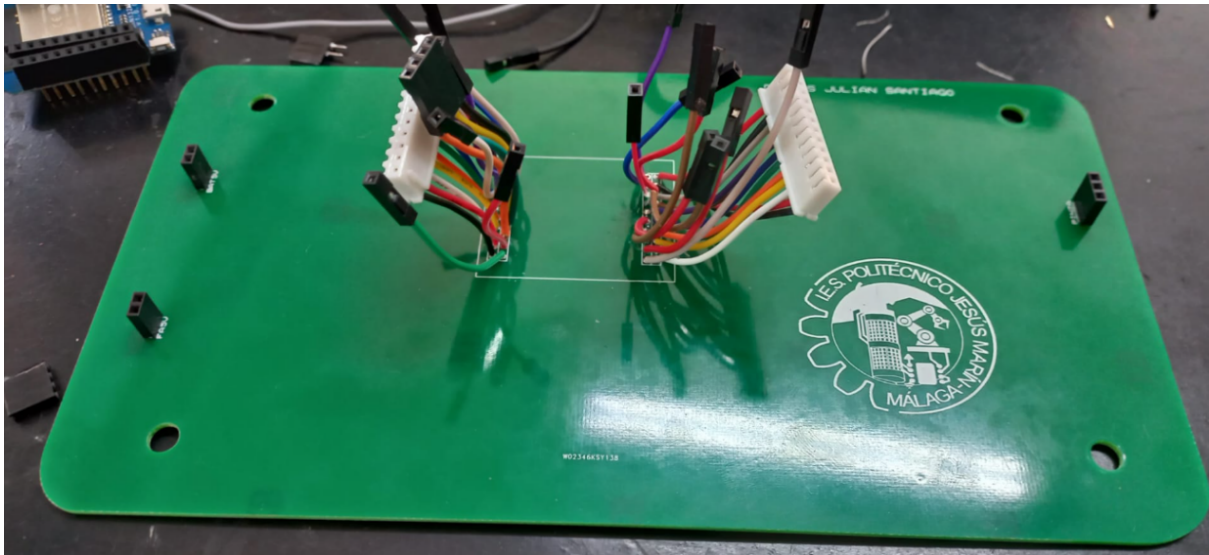


La placa tiene un tamaño demasiado grande y los pads de los componentes no tienen la mismas dimensiones que los componentes reales. Otro fallo del que muy probablemente aprenderemos: por mucho que se vea bien y parezca que funcione, hay que comprobar el trabajo de los compañeros todas las veces que haga falta.

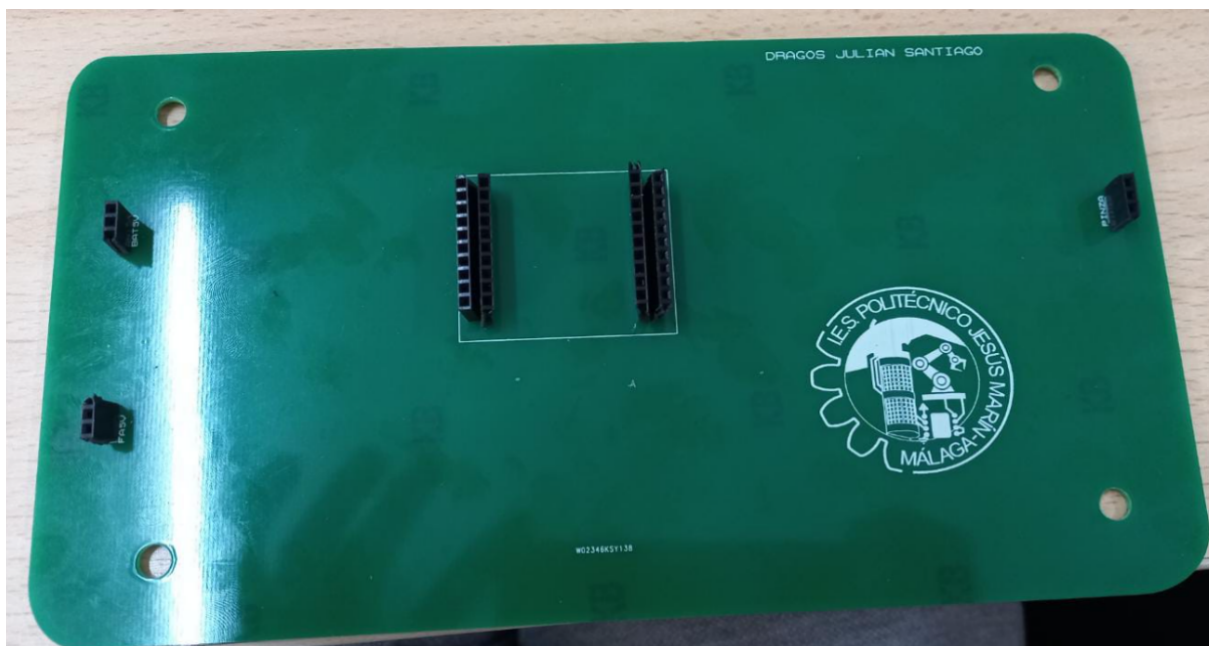
Las prisas por la creencia de falta de tiempo nos jugaron una mala pasada esta vez. De volver a repetir de nuevo este proyecto, seguramente cometería los mismos errores, pero a la tercera vez segurísimo que aprendería algo de todo esto.

5. MONTAJE FINAL

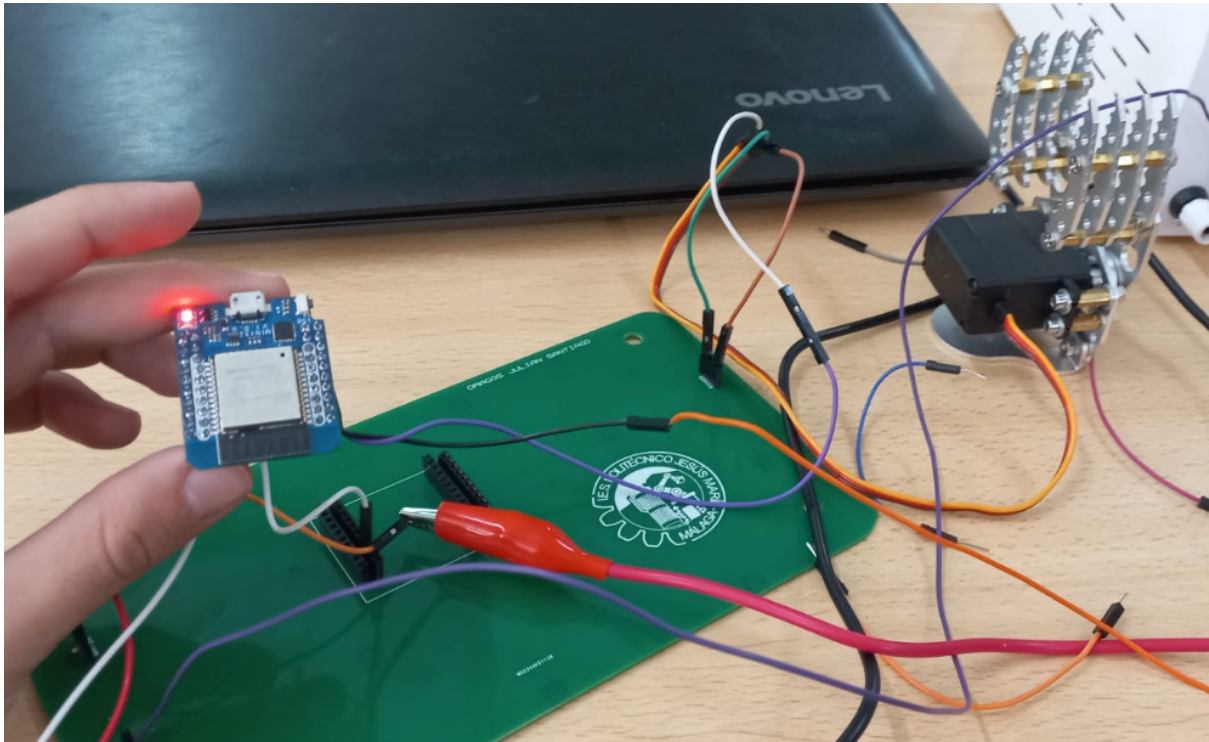
Debido a los problemas con las dimensiones de los componentes en las placas he tenido que realizar un apaño para poder conectar el mini ESP32. En primera instancia decidí realizar una soldadura de cables macho hembra para luego conectar el lado hembra a los pines de la ESP. El resultado no fue satisfactorio debido básicamente a la fragilidad de los cables. Repetimos el mismo proceso en otra placa.



Como alternativa, igual que en la maqueta tiflológica, hemos soldado nuevos conectores en los pads donde iría la mini ESP32 con el tamaño adecuado.



Para conectar la esp 32 conectamos los cables en los conectores y de ahí a los pines, ya que el tamaño real del microcontrolador no encaja con el diseño para la placa.

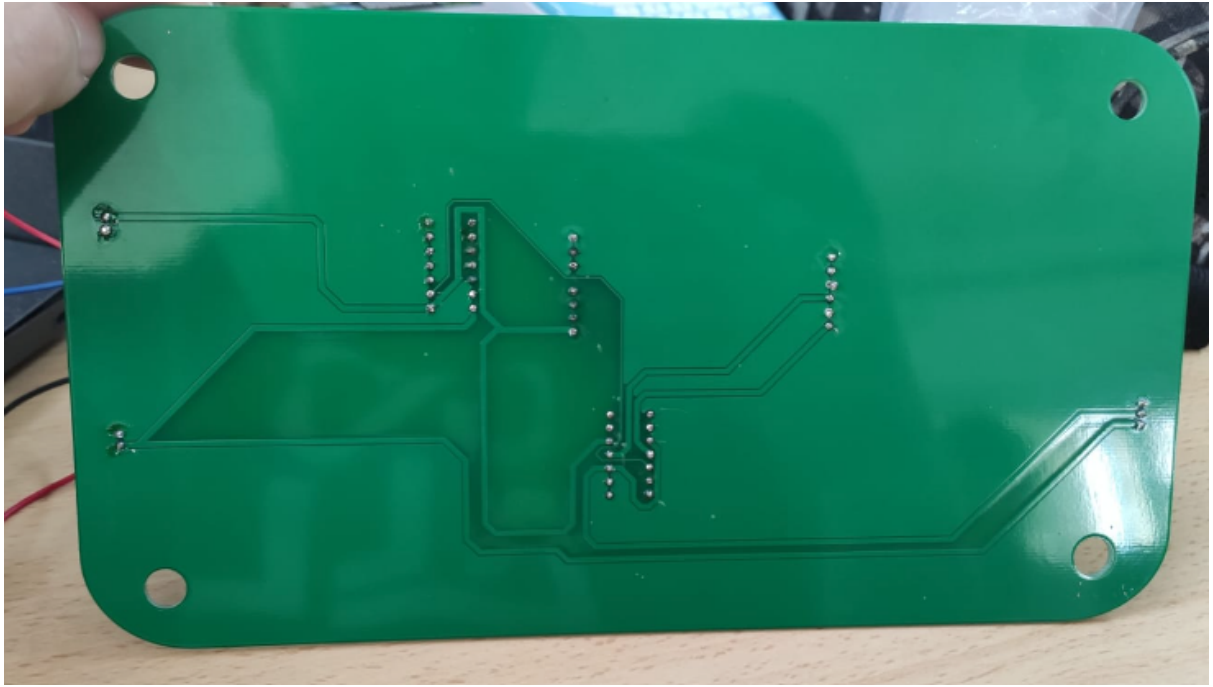


Hemos tenido un problema en el diseño de las pistas, debido que a un error del programa, la pista que proporciona la señal a la pinza no aparece, por lo que debemos conectarla directamente al pin del ESP32. Un error parecido pasó con la maqueta tiflológica: hay varias pistas que en el archivo Gerber pre-impresión están, pero luego no están al llegar de China. Otra cosa que hemos descubierto es que en el mini ESP32 solo posee 3 pines de PWM cosa que hemos corregido. Una vez conectado correctamente, pese a los fallos de diseño y conexionado hemos logrado que funcione de la manera que deseábamos. Seguiremos intentando mejorar su funcionamiento corrigiendo los fallos en las siguientes clases.

6. Funcionamiento del montaje. Errores finales.

<https://drive.google.com/file/d/1CFRVUZrJm73-CASAIcImivi3hbhWgloS/view?usp=sharing>

Respecto a la placa con componentes conseguimos realizar la soldadura correctamente, teniendo más cuidado. Algo que no tuvimos en cuenta es el diseño para los pines de salida del generador del pulso, en vez de crear 3 salidas solo implementamos uno. Eso limitó en un 66% el funcionamiento correcto del conjunto.





Pese al error de las pistas el funcionamiento es correcto (cumple con su cometido), con el inconveniente de que el cambio de estado de apertura y cierre de la pinza hay que ajustarlo manualmente mediante cambio de voltaje con el potenciómetro.